# Waste in the City

*An Agent-Based Model of Urban Waste Dynamics*

Symbolic AI and Rule-Based Agents (SARA)
OTH Amberg-Weiden — Summer Semester 2026
Final Project Report

## 1. Introduction

This report describes the Waste in the City project: a grid-based Agent-Based Model (ABM) implemented in Python with the Mesa framework. The simulation studies how city structure, the movement habits of residents and tourists, the placement of bins, and the strategy used by cleaning services together shape the emergent distribution of litter on city streets. The focus is on emergent, city-wide behaviour that arises from many simple rule-based agents interacting on a 2D grid.

**Project goals:**

- Model a city as a walkable grid with streets, buildings, public areas, attractions, bins, containers, and a disposal point.
- Implement five rule-based agent types whose interactions cause waste to appear, accumulate, and be cleared.
- Use classical graph-search algorithms (BFS, multi-target BFS, A*) for navigation, satisfying the Symbolic AI requirement.
- Compare four cleaning strategies, including a creative extension: a decaying “waste-observation heatmap”.
- Produce statistics and visualisations (live animation + metric plots) for analysis and reporting.

## 2. Project Structure

The project is organised as a small Python package:

```
final_project/
├── README.md             # Documentation
├── requirements.txt      # Python dependencies
├── config.py             # All tunable parameters
├── pathfinding.py        # BFS / multi-target BFS / A*
├── agents.py             # Five agent classes
├── city_model.py         # Mesa Model: layout, scheduler, DataCollector
```

```
├── visualize.py          # Matplotlib renderer
├── experiments.py        # Slide-46 comparison experiments
├── run.py                # Command-line entry point
├── city_layouts/
│   ├── default.txt       # 30x30 ASCII city map
│   └── README.md         # Legend for layout files
└── results/              # Output: plots + CSVs
```

| File | Responsibility |
|---|---|
| **config.py** | Single source of truth for tunable parameters: grid size, populations, waste probabilities, bin capacities, heatmap decay, etc. |
| **pathfinding.py** | Graph-search utilities (bfs_shortest_path, bfs_nearest, astar_path) operating on the implicit graph of walkable cells. |
| **agents.py** | Five mesa.Agent subclasses for locals, tourists, cleaners, bins, and transporters. |
| **city_model.py** | WasteCityModel that parses the layout, places infrastructure, spawns mobile agents, and drives Mesa's scheduler & DataCollector. |
| **visualize.py** | Matplotlib renderer producing the live animation, static snapshots, and the four-panel metrics figure. |
| **experiments.py** | Batch-runs that reproduce the slide-46 sweeps (cleaner strategies, tourist density, bin/cleaning capacity). |
| **run.py** | Command-line entry point: choose strategy, populations, steps, animation/headless mode, save GIFs/PNGs, or trigger experiments. |
| **city_layouts/default.txt** | ASCII map of the default 30x30 city used by the simulation. |

## 3. Installation and Quick Start

Requires Python 3.10+. Install dependencies and run:

```
python -m venv .venv
.venv\Scripts\activate          # Windows
pip install -r requirements.txt

# Default simulation with live animation
python run.py

# Headless 500-step run, save metrics PNG
python run.py --no-animate --steps 500 --save-metrics metrics.png

# Save the live animation as a GIF
```

```
python run.py --save-animation city.gif --steps 200

# Try the heatmap (creative extension) cleaner
python run.py --strategy heatmap

# Run the full experiment battery
python run.py --experiments
```

Dependencies (requirements.txt):

```
mesa>=2.1,<3.0
numpy>=1.23
pandas>=1.5
matplotlib>=3.6
```

## 4. The City Grid

The world is described by an ASCII map (see city_layouts/default.txt). Each character corresponds to one cell of the grid and is interpreted by city_model.WasteCityModel._scan_layout when the model is built.

| Symbol | Meaning | Walkable? |
|---|---|---|
| **#** | Wall / building | No |
| **.** | Street / walkable path | Yes |
| **P** | Public area (waste accumulates more easily) | Yes |
| **A** | Attraction (tourists gravitate here) | Yes |
| **B** | Dust bin (small, capacity = BIN_CAPACITY) | Yes (agent on cell) |
| **C** | Dust container (large, capacity = CONTAINER_CAPACITY) | Yes (agent on cell) |
| **D** | Disposal point (transporter destination) | Yes |

Default layout (30x30) — abridged view:

```
D....B.....B.....B.....B......
.#####.#####.#####.#####.#####
.....B.................B......
...           (building blocks)
.....B......C..........B......
.#####.#####.PPPP#.#####.#####
.#####.#####.PAPP#.#####.#####   <- central plaza with attractions
.#####.#####.PPAP#.#####.#####
.....B............C....B......
```

```
...
```

Movement is restricted to the 4-connected grid graph (no diagonals). Agents rely on the graph-search utilities in pathfinding.py to plan routes between any two walkable cells, fulfilling the requirement to use graph-search algorithms where it makes sense.

## 5. Rule-Based Agents

Five agent types live on the grid. Each is implemented as a subclass of mesa.Agent and exposes a step() method that defines what happens on every simulation tick.

### 5.1 LocalHumanAgent (Resident)

- State: home cell, work cell, current target, cached BFS path.
- Behaviour: commutes home <-> work each day. When it arrives at the target, the target flips to the other endpoint.
- Movement: uses bfs_shortest_path between current cell and the current target; falls back to a random walk if the path is blocked.
- Waste: with probability LOCAL_WASTE_PROB (default 0.05) the local wants to dispose of waste. It first runs a multi-target BFS (bfs_nearest) to find a non-full bin within HUMAN_BIN_SEARCH_RADIUS (default 5). If a bin is reachable, the waste goes into the bin (metric waste_into_bins increments). Otherwise the waste is dropped on the ground at the current cell.

### 5.2 TouristAgent (Visitor)

- State: target attraction, cached BFS path.
- Behaviour: drifts toward attraction cells (A). With small probability per step the target is re-randomised to another attraction, modelling unpredictable sightseeing.
- Movement: 30% of steps are pure random walks; the rest follow the BFS path to the chosen attraction.
- Waste: with the higher probability TOURIST_WASTE_PROB (default 0.15) tourists drop waste directly on the ground -- they are modelled as less likely to use bins than locals.

### 5.3 CleaningServiceAgent (Street Cleaner)

- Priority action: if standing on a cell with ground waste, the cleaner removes it immediately and increments waste_cleaned.
- Otherwise picks a target according to its strategy (see Section 6) and walks one step along the BFS path toward that target.
- If no target is found, falls back to a random walk so it never freezes.

### 5.4 DustBinAgent (Bin / Container)

- Static infrastructure with a fixed capacity (BIN_CAPACITY for small bins, CONTAINER_CAPACITY for large containers).

- add_waste(amount) returns leftover that did not fit; each leftover increments overflow_count.
- fill_ratio = load / capacity is exposed for metrics and for transporter decision-making.
- empty() resets the load and returns the amount removed (used by the transporter when collecting).
- step() is a no-op: bins are passive but still scheduled so that Mesa treats them uniformly.

### 5.5 DustTransporterAgent (Truck)

- Bases out of a depot (disposal point D). Carries a cargo of collected waste between bins and the depot.
- Picks the closest bin whose fill_ratio >= TRANSPORTER_FULL_THRESHOLD (default 0.7) using multi-target BFS, drives there, empties it.
- When at the depot with non-zero cargo, dumps the cargo and increments waste_disposed -- this is the only path that removes waste from the system permanently.

## 6. Cleaning Strategies

The CleaningServiceAgent supports four interchangeable rule-based strategies. Selecting the strategy is the central experimental knob of the project (see Section 9).

| Strategy | Behaviour |
|---|---|
| **nearest_waste** | Multi-target BFS from the cleaner's cell to the closest known ground-waste cell. Greedy reactive baseline. |
| **random_patrol** | Pick any walkable neighbour at random each step. Pure random walk; no global planning. |
| **fixed_route** | Cycle through a preset list of waypoints (bin cells by default). Models a scheduled patrol. |
| **heatmap** | Creative extension. Heads for the cell with the highest decaying heatmap value -- the strongest hotspot the city has 'learned' about over time. |

## 7. Creative Extension: Decaying Heatmap

The simulation maintains a 2D heatmap (heatmap[y][x]) shared across the model. Whenever waste appears at a cell (model.add_ground_waste), heatmap[y][x] is incremented by HEATMAP_INCREMENT (default 1.0). Every model step the entire heatmap is multiplied by HEATMAP_DECAY (default 0.97), so:

- Persistent hotspots (e.g. tourist attractions, public squares) stay strong because they are repeatedly reinforced.
- Old, isolated waste events fade exponentially over time.

- model.heatmap_argmax() returns the strongest hotspot, which the heatmap cleaner uses as its current goal.

This adds memory and prediction to the otherwise reactive cleaning service: cleaners can pre-position themselves around chronic problem areas even when individual pieces of waste have just been produced and are not yet visible to the greedy nearest-waste heuristic.

## 8. Graph Search and Pathfinding

The city is treated as an implicit graph: nodes are walkable grid cells, and undirected edges connect 4-neighbours that are both walkable. pathfinding.py exposes three reusable functions:

| Function | Algorithm | Used By |
|---|---|---|
| **bfs_shortest_path(start, goal, walkable)** | Breadth-First Search. Optimal in number of hops on an unweighted grid. | Locals, tourists, fixed_route / heatmap cleaners, transporter. |
| **bfs_nearest(start, targets, walkable)** | Multi-target BFS: returns the closest target and a path in one sweep. | Locals (closest non-full bin), nearest_waste cleaner, transporter. |
| **astar_path(start, goal, walkable)** | A* with Manhattan distance heuristic; admissible and optimal on this grid. | Available for goal-directed routing where a heuristic helps. |

These algorithms satisfy the explicit lecture requirement to use graph search algorithms where it makes sense and keep agent code concise: each agent simply calls the appropriate utility.

## 9. The Mesa Model: WasteCityModel

city_model.WasteCityModel is the orchestrator. Its constructor:

- Seeds the random number generator (config.RANDOM_SEED) for reproducibility.
- Parses the ASCII layout into a 2D character grid (cell_types).
- Creates a mesa.space.MultiGrid (no torus) so multiple agents can share a cell.
- Uses mesa.time.RandomActivation: every step each agent acts once in random order.
- Calls _scan_layout() to instantiate B/C/D infrastructure and remember walkable cells, attractions, and disposal points.
- Spawns NUM_LOCALS, NUM_TOURISTS, NUM_CLEANERS and NUM_TRANSPORTERS with non-clashing unique IDs.
- Configures a mesa.datacollection.DataCollector whose model reporters expose the metrics described in Section 10.

Per-step update (model.step()):

```
1. datacollector.collect(self)        # sample metrics first
2. heatmap *= HEATMAP_DECAY            # creative extension decay
```

```
3. schedule.step()                    # all agents act, random order
```

## 10. Metrics

The DataCollector samples the following each step. The result is a tidy pandas DataFrame (one row per step) accessible via model.datacollector.get_model_vars_dataframe().

| Metric | Definition | Interpretation |
|---|---|---|
| **GroundWaste** | Sum of model.ground_waste.values(). | Total litter currently lying on streets. |
| **OverflowingBins** | Count of bins/containers with load >= capacity. | Indicator of bin saturation. |
| **AvgBinFill** | Mean of fill_ratio across all bins. | Proxy for the 'average waste per district'. |
| **WasteCleaned** | Cumulative units removed by cleaners. | Throughput of the cleaning service. |
| **WasteDisposed** | Cumulative units delivered to the depot. | Throughput of the transporter, the only true sink. |
| **WasteIntoBins** | Cumulative units locals put into bins. | Proxy for responsible disposal behaviour. |

Per-agent counters (cleaned_units on cleaners, bins_emptied on transporters, overflow_count on bins) are stored as attributes for ad-hoc analysis after a run.

## 11. Visualisations

All visual output is produced by visualize.py using Matplotlib. The module deliberately avoids Mesa's web visualisation server so the project remains stable across Mesa minor releases.

### 11.1 Static Snapshot — draw_city(model)

Renders the current state of the simulation on a single axis. The image is composed of three layers drawn on top of each other:

**Layer 1 — Static cell background (imshow):**

| Cell | Code | Colour | Hex |
|---|---|---|---|
| **Building (#)** | 0 | Dark grey | #3a3a3a |
| **Street (.)** | 1 | Light grey | #dddddd |
| **Public area (P)** | 2 | Light green | #c5e1a5 |
| **Attraction (A)** | 3 | Yellow | #fff176 |
| **Bin (B)** | 4 | Blue | #42a5f5 |
| **Container (C)** | 5 | Dark blue | #1565c0 |
| **Disposal (D)** | 6 | Brown | #8d6e63 |

**Layer 2 — Ground waste:**

Cells with litter are overlaid as red squares. The marker size scales with the amount of waste at that cell (size = 12 * amount), so heavily littered cells become visually dominant. Each bin's current load is rendered as a small white number on top of the bin cell, providing an instant visual readout of fill levels.

**Layer 3 — Mobile agents:**

| Agent | Marker | Colour |
|---|---|---|
| **LocalHumanAgent** | Circle (o) | Dark green (#1b5e20) |
| **TouristAgent** | Triangle (^) | Orange (#ef6c00) |
| **CleaningServiceAgent** | Diamond (D) | Purple (#6a1b9a) |
| **DustTransporterAgent** | Square (s) | Dark red (#b71c1c) |

The plot title is updated to include the current step and the total ground-waste sum, so a single still frame already conveys progress. A side legend decodes the static-cell colour palette.

## 11.2 Live Animation — animate(model, steps)

- Built on matplotlib.animation.FuncAnimation. Every frame: clear the axes, advance the model with model.step(), and redraw via draw_city.
- blit=False because legend, scatter sizes, and bin numbers all change between frames.
- Default frame interval 80 ms. If save_path is given, the animation is written as a GIF using Pillow (no ffmpeg required); otherwise an interactive Matplotlib window opens.
- Lets the viewer watch how locals commute, tourists meander, waste piles grow, bins fill, and cleaners react in real time.

## 11.3 Metrics Figure — plot_metrics(model)

A 2x2 figure (figsize 11x7) summarising the entire run:

| Subplot | Series | Colour / Style | Question Answered |
|---|---|---|---|
| **Top-left** | GroundWaste | Red line | How dirty are the streets over time? |
| **Top-right** | OverflowingBins | Orange line | How often does the bin network saturate? |
| **Bottom-left** | AvgBinFill | Blue line | What is the typical district-level fill level? |
| **Bottom-right** | WasteCleaned, WasteDisposed, WasteIntoBins (cumulative) | Three lines on shared axes | Where does the waste actually flow? |

All four subplots share the same x-axis (simulation step) and use a light grid. The cumulative throughput chart is the main diagnostic of how well the cleaning + disposal pipeline is keeping up: when WasteDisposed grows in step with WasteCleaned, the system is in a healthy steady state; otherwise waste is accumulating in bins or on the ground.

### 11.4 Experiment Comparison Plots

experiments.run_all() executes the slide-46 sweeps and writes four comparison figures (and the underlying CSVs) into the results/ folder:

| File | Compares | Insight |
|---|---|---|
| **compare_strategies_ground_waste.png** | GroundWaste(t) for the four cleaner strategies on the same world. | Which strategy keeps streets cleanest? Heatmap usually beats random_patrol and fixed_route, and is competitive with nearest_waste. |
| **compare_strategies_overflow.png** | OverflowingBins(t) per strategy. | Which strategy best prevents bin saturation indirectly (by cleaning nearby ground waste before bins are pressured)? |
| **tourists_ground_waste.png** | GroundWaste(t) for low (2) vs. high (30) tourist density. | Quantifies how strongly tourist density drives litter production -- the dominant exogenous factor in the model. |
| **bin_density_overflow.png** | OverflowingBins(t) for full vs. reduced cleaning capacity. | Shows how fragile the system becomes when cleaners and transporters are removed. |

Each plot uses one Matplotlib line per run, the run name as the legend entry, and a transparent grid (alpha=0.3). The raw CSVs allow any further statistical analysis (e.g. mean/variance over multiple seeds).

## 12. Configuration Reference (config.py)

| Parameter | Default | Effect |
|---|---|---|
| **GRID_WIDTH / GRID_HEIGHT** | 30 / 30 | Grid dimensions (must match the layout). |
| **DEFAULT_LAYOUT_FILE** | city_layouts/default.txt | Path to the ASCII map. |
| **NUM_LOCALS / NUM_TOURISTS** | 25 / 10 | Population of mobile humans. |
| **NUM_CLEANERS / NUM_TRANSPORTERS** | 3 / 1 | Population of cleaning service and trucks. |
| **LOCAL_WASTE_PROB** | 0.05 | Probability per step a local produces waste. |
| **TOURIST_WASTE_PROB** | 0.15 | Probability per step a |

| | | |
|---|---|---|
| | | tourist produces waste. |
| **HUMAN_BIN_SEARCH_RADIUS** | 5 | Maximum BFS-distance a local will travel to use a bin. |
| **BIN_CAPACITY / CONTAINER_CAPACITY** | 10 / 50 | Storage capacity of small bins and large containers. |
| **TRANSPORTER_FULL_THRESHOLD** | 0.7 | fill_ratio above which a bin is worth a transporter trip. |
| **CLEANER_STRATEGIES** | tuple of 4 | Whitelist of valid cleaner strategies. |
| **DEFAULT_CLEANER_STRATEGY** | nearest_waste | Strategy used when no override is provided. |
| **HEATMAP_DECAY / HEATMAP_INCREMENT** | 0.97 / 1.0 | Decay factor and reinforcement step for the creative extension. |
| **NUM_STEPS** | 300 | Default simulation length. |
| **RANDOM_SEED** | 42 | Seed for reproducible runs (None = non-deterministic). |
| **TRANSPORTER_INTERVAL** | 25 | Scheduling interval for transporter dispatch. |

## 13. Mapping to Lecture Slides 41–48

| Slide Requirement | Where it lives in the project |
|---|---|
| **Grid world with streets / buildings / public areas** | city_layouts/default.txt + WasteCityModel._scan_layout |
| **Local humans with daily patterns + occasional waste** | agents.LocalHumanAgent |
| **Tourists, less predictable, prefer attractions, more waste** | agents.TouristAgent |
| **Cleaning service with rule-based strategies** | agents.CleaningServiceAgent + config.CLEANER_STRATEGIES |
| **Bins / containers with capacity & overflow** | agents.DustBinAgent |
| **Dust transporters going to disposal** | agents.DustTransporterAgent |
| **Use of graph search algorithms** | pathfinding.py (BFS, multi-target BFS, A*) |
| **Implementation in Mesa** | WasteCityModel + MultiGrid + RandomActivation + DataCollector |
| **Experiments on slide 46 (strategies, density, capacity)** | experiments.py + run.py --experiments |
| **Statistics over simulation steps** | WasteCityModel.datacollector + visualize.plot_metrics |
| **Creative extension (slide 47)** | Decaying heatmap + heatmap cleaner strategy |

## 14. End-to-End Walk-through of a Single Step

Putting the pieces together, this is what happens during one tick:

- 1. The DataCollector samples GroundWaste, OverflowingBins, AvgBinFill, WasteCleaned, WasteDisposed and WasteIntoBins.
- 2. The heatmap is multiplied by HEATMAP_DECAY (=0.97).
- 3. The scheduler activates every agent in random order:
- - Locals: pick BFS path to home/work, walk one cell, then with probability 0.05 either deposit waste in the nearest non-full bin within radius 5 or drop it on the ground.
- - Tourists: with probability 0.05 retarget a random attraction; 30% of steps are random walks, otherwise BFS toward the target. With probability 0.15 drop waste on the ground.
- - Cleaners: if standing on waste, clean it; otherwise walk one step toward the target chosen by the active strategy.
- - Transporter: if at the depot with cargo, dispose of it; if at a target bin, empty it; otherwise BFS toward the next bin (or back to depot).
- - Bins: passive (their state changes when written by other agents).
- 4. Whenever ground waste is added, model.add_ground_waste also increments the heatmap, feeding the creative extension.

## 15. Notes, Caveats and Limitations

- Random activation means per-step ordering varies; reproducibility is provided exclusively through RANDOM_SEED.
- BFS path caching is intentionally simple: paths are recomputed whenever the start cell does not match the cached head, which is good enough for grids of this size.
- For speed, locals 'use' a bin without physically walking to it when one is reachable within HUMAN_BIN_SEARCH_RADIUS. This is an abstraction documented in agents.LocalHumanAgent._maybe_drop_waste.
- The visualisation deliberately uses Matplotlib, not Mesa's web server, to remain compatible across Mesa 2.x patch releases.
- Pinned to Mesa 2.x. Migrating to Mesa 3 will require updating the imports in agents.py and city_model.py.

## 16. Conclusion

The Waste in the City project demonstrates how a small set of clearly defined rule-based agents, navigating a grid via classical graph search, can produce rich, city-scale waste dynamics. The Mesa framework provides the scheduling and metrics plumbing; the Matplotlib renderer turns the simulation into a readable live animation; the experiment battery answers the slide-46 questions in a reproducible, scriptable way; and the decaying-heatmap creative extension shows how a tiny addition of memory can lift a reactive cleaning policy into a predictive one.